

Depth Pseudo-Color Image

Depth Pseudo-Color Image

1. Course Content
2. Program Startup
3. Source Code Analysis

1. Course Content

Run example programs, convert black and white depth images to pseudo-color images by subscribing to depth color image topics and using OpenCV image processing.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

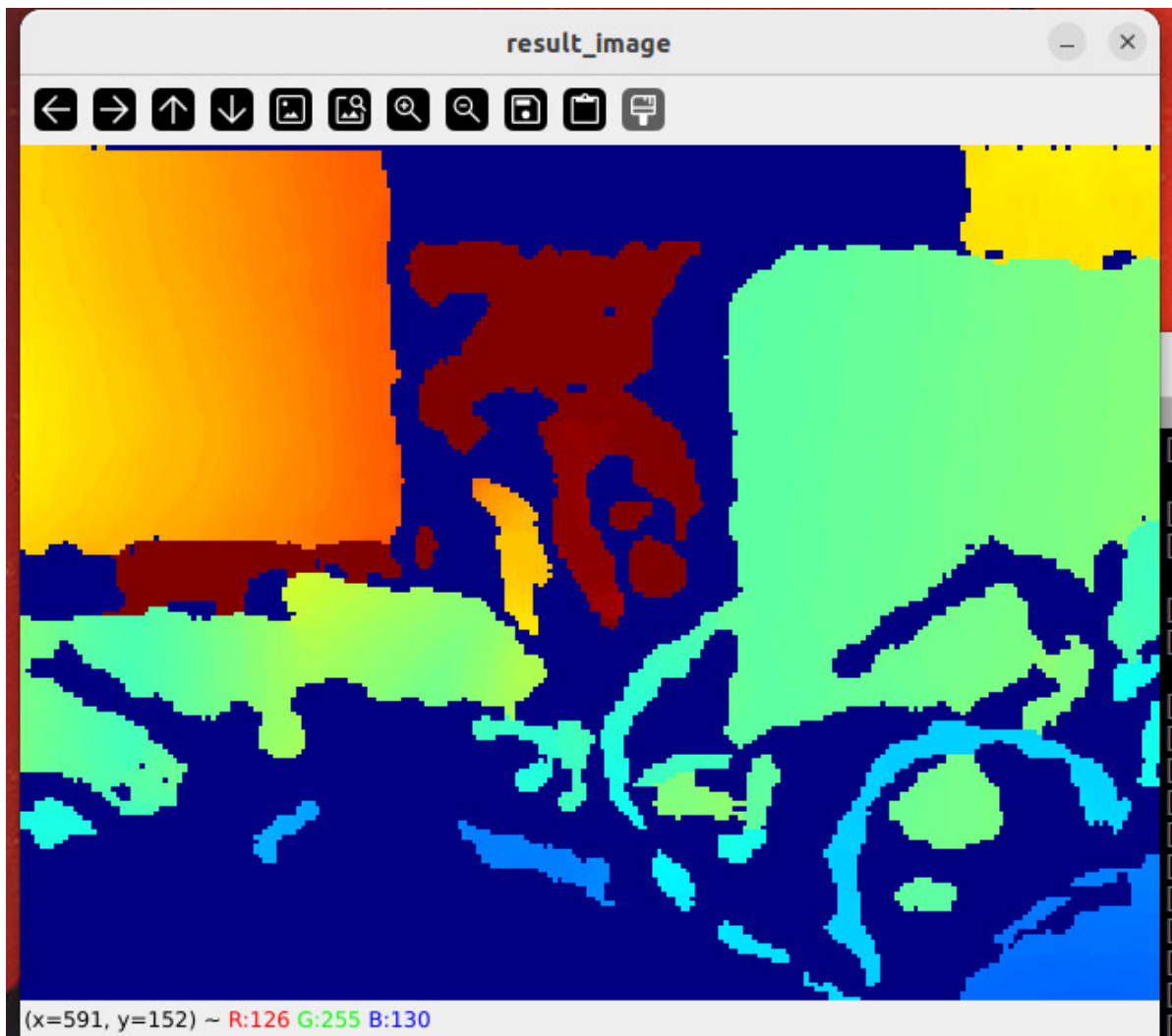
Open a terminal inside the container,

```
exec_m3
```

- Enter command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py
```

After startup, as shown in the figure below:



Different colors represent differences in depth information.

3. Source Code Analysis

- Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/GetDepthColor.py
```

- Import required library files

```
from cv_bridge import CvBridge
import cv2
from rclpy.node import Node
import rclpy
from sensor_msgs.msg import Image
```

- Define depth image decoding format

```
encoding = ['16UC1', '32FC1']
```

- Define subscriber for depth image topic

```
self.sub_depth =  
self.create_subscription(Image, "/camera/depth/image_raw", self.get_DepthImgCallback, 100)
```

- Define self.depth_bridge for converting message format to image format that OpenCV can process

```
self.depth_bridge = CvBridge()
```

- Convert image data message to image

```
depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame, encoding[1])
```

- Call OpenCV image processing function cv2.applyColorMap to convert depth map

```
depth_to_color_image = cv2.applyColorMap(cv2.convertScaleAbs(depth_image,  
alpha=1.0), cv2.COLORMAP_JET)
```

cv2.convertScaleAbs: Performs **linear transformation** on image data and converts it to **8-bit unsigned integer (uint8)** format, with scaling factor as the scaling factor.

cv2.applyColorMap: A function that converts **single-channel grayscale images** to **pseudo-color images** to enhance visualization effects through color mapping. This parameter has two variables: the first variable is a single-channel 8-bit or floating-point image (grayscale image), and the second variable is the color mapping table, with the following optional values:

cv2.COLORMAP_AUTUMN	# Red-Orange-Yellow
cv2.COLORMAP_BONE	# Black-White (grayscale with blue tint)
cv2.COLORMAP_HSV	# HSV color space
cv2.COLORMAP_JET	# Blue-Cyan-Yellow-Red (classic heat map)
cv2.COLORMAP_WINTER	# Blue-Green
cv2.COLORMAP_RAINBOW	# Rainbow colors (Red-Purple)
cv2.COLORMAP_OCEAN	# Dark Blue-Light Blue
cv2.COLORMAP_SUMMER	# Green-Yellow
cv2.COLORMAP_SPRING	# Magenta-Yellow
cv2.COLORMAP_COOL	# Cyan-Magenta
cv2.COLORMAP_HSV	# HSV color space cycle
cv2.COLORMAP_PINK	# Pink tones
cv2.COLORMAP_PARULA	# MATLAB Parula style
cv2.COLORMAP_VIRIDIS	# Modern scientific color scale (Green-Purple)
cv2.COLORMAP_INFERNO	# Black-Red-Yellow-White (high contrast)
cv2.COLORMAP_PLASMA	# Purple-Red-Yellow
cv2.COLORMAP_MAGMA	# Black-Red-White
cv2.COLORMAP_TWILIGHT	# Blue-Pink-White gradient